Ministerul Educaţiei şi Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică şi Microelectronică

Departamentul Ingineria Software şi Automatică

Report on Laboratory Work No. 6

Inside the xv6 File System

Operating Systems

Done by: st. gr. FAF-241, Caţ Chiril

Checked by: asist. univ. Reiţman Patricia

Chişinău – 2026

Contents

1 Purpose and environment **1**

2 Directory entries and file types **1**

3 Metadata and hard links **2**

4 File system limits **3**

5 Recursive find **4**

6 Handout questions and answers **5**

6.1 Part 1: what xv6 shows 5

6.2 Part 2: mystat and hard links 5

6.3 Part 3: file system limits 6

7 Conclusion **6**

1 Purpose and environment

I inspected the xv6 file system's directory entries, inodes, hard links, and disk layout, then ran two user programs: `mystat`, which reports metadata for paths, and `find`, which walks directories recursively. I used the `os/xv6-riscv` tree with the local RISC-V toolchain and QEMU on 26 October 2026. I rebuilt the file system image with `make clean && make fs.img`. The selected command output below is also recorded in the accompanying notes. Inode numbers depend on the programs present in the image.

2 Directory entries and file types

`kernel/stat.h` defines type 1 as a directory (`T_DIR`), type 2 as a regular file (`T_FILE`), and type 3 as a device (`T_DEVICE`). The root has inode 1, as defined by `ROOTINO`. Its `.` and `..` both refer to inode 1 because the root has no parent directory above it.

```
$ ls .
.              1 1 1024
..             1 1 1024
README         2 2 2441
... (other built-in programs)
console        3 27 0
$ mkdir a
$ ls a
.              1 28 32
..             1 1 1024
```

An empty `a` has two entries, `.` (inode 28) and `..` (inode 1). Its 32-byte size follows from two 16-byte `dirent` records. Each record stores a 2-byte inode number and a 14-byte name.

3 Metadata and hard links

`mystat` calls `stat()` for each argument. It prints the file type, inode number, link count, and size; an invalid path is reported on standard error without stopping subsequent arguments.

```
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 2, size 1024 bytes
console: device, inode 27, links 1, size 0 bytes
$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes
$ rm README
$ mystat r2
r2: file, inode 2, links 1, size 2441 bytes
$ cat r2 | wc
48 336 2441
```

Creating `r2` added a second directory entry for inode 2, so the link count became 2 while inode and size stayed the same. Removing `README` reduced the count to 1. The content remained readable through `r2`. The root's link count was 2 here because `mkdir a` had already added a child directory.

In `kernel/sysfile.c`, `sys_unlink` performs `ip->nlink--;` and `iupdate(ip);`. When the count becomes zero and the final in-memory reference is released, `iput()` in `kernel/fs.c` invokes `itrunc(ip)` to free the data blocks. The on-disk `struct dinode` contains `type`, `major`, `minor`, `nlink`, `size`, and `addrs[NDIRECT + 1]`: twelve direct block addresses and one indirect block address. It has no permissions or symbolic-link field.

4 File system limits

The constants in `kernel/fs.h` are `BSIZE=1024` and `NDIRECT=12`. The indirect block holds `NINDIRECT=BSIZE/sizeof(uint)=256` addresses. The directory name limit is `DIRSIZ=14`, so a longer name is truncated to 14 bytes:

```
$ mkdir averyveryverylongname
$ ls .
averyveryveryl 1 29 32
```

The largest file is

$$\left(12 + \frac{1024}{4}\right) \times 1024 = 274{,}432 \text{ bytes} = 268 \text{ KiB}.$$

This uses twelve direct blocks and 256 block addresses in one indirect block. The image builder printed:

```
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks
   1953 total 2000
```

The disk has 2000 blocks in total: 31 for the log, 13 for inodes, 1953 for data, and one each for the boot block, superblock, and bitmap. Thus the metadata count is $2+31+13+1 = 47$.

5 Recursive find

`find` checks that its first argument is a directory, reads each `dirent`, ignores free entries and `./..`, compares the last name component with `strcmp`, and recursively visits child directories. It closes each opened directory. Its path buffer is checked before appending a child name.

```
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ find . r2
./a/b/r2
./a/r2
./r2
$ find a b
a/b
```

The three paths match by last component. The root-level `r2` is the remaining hard link to the original `README`; the two files created with `echo` are separate files. An additional link check confirmed that `find` reports a linked name and `mystat` identifies its inode:

```
$ ln cat r3
$ find . r3
./r3
$ mystat cat r3
cat: file, inode 7, links 2, size 37344 bytes
r3: file, inode 7, links 2, size 37344 bytes
```

6 Handout questions and answers

The following are the required Observe questions from the Lab 6 handout, with the answers derived from the xv6 sources and the QEMU output above.

6.1 Part 1: what xv6 shows

1. **Question:** What do types 1, 2 and 3 mean (`kernel/stat.h`)?

   Answer: Type 1 (`T_DIR`) is a directory, type 2 (`T_FILE`) is a regular file, and type 3 (`T_DEVICE`) is a device.

2. **Question:** Which inode number does the root directory have, and why are `.` and `..` the same there?

   Answer: The root has inode 1 (`ROOTINO`). Both names refer to inode 1 because the root has no parent directory above itself.

3. **Question:** An empty directory `a` has size 32. A directory entry has a fixed size. How big is one entry, and which two entries does `a` contain?

   Answer: One entry is 16 bytes: a 2-byte inode number and a 14-byte name. The two entries are `.` (`a` itself) and `..` (its parent).

6.2 Part 2: mystat and hard links

1. **Question:** Paste the outputs after `ln` and after `rm`. What changed and what did not?

   Answer: The measured output was:

   ```
   $ ln README r2
   $ mystat README r2
   README: file, inode 2, links 2, size 2441 bytes
   r2: file, inode 2, links 2, size 2441 bytes
   $ rm README
   $ mystat r2
   r2: file, inode 2, links 1, size 2441 bytes
   ```

   Both names initially shared inode 2 and its 2441-byte data. `ln` increased the link count to two. Removing `README` removed one name and reduced the count to one; the inode, size, and content readable through `r2` did not change.

2. **Question:** Find the line in `kernel/sysfile.c` (`sys_unlink`) that decrements the link count. When are the data blocks freed?

   Answer: The line is `ip->nlink--;`, followed by `iupdate(ip);`. When the link count is zero and the last in-memory reference is released, `iput()` in `kernel/fs.c` calls `itrunc(ip)` to free the blocks.

3. **Question:** xv6 has no symbolic links and no permissions. Look at `struct dinode` in `kernel/fs.h`: which fields does an xv6 inode have?

   Answer: It has `type`, `major`, `minor`, `nlink`, `size`, and `addrs[NDIRECT + 1]`. The address array holds twelve direct block addresses and one indirect address.

6.3 Part 3: file system limits

1. **Question:** What does `ls` show for `averyveryverylongname`, and which constant explains it?

   Answer: It showed `averyveryveryl`. That is the first 14 characters; `DIRSIZ=14` limits directory-entry names.

2. **Question:** Compute the largest file xv6 can hold: 12 direct blocks plus one indirect block that holds block numbers of 4 bytes each, blocks of `BSIZE` bytes. Show the calculation.

   Answer: With `BSIZE=1024`, the indirect block holds $1024/4 = 256$ addresses. The maximum is $(12 + 256) \times 1024 = 274{,}432$ bytes, or 268 KiB.

3. **Question:** From the `nmeta` line: how many blocks does the whole disk have, and how many are for the log, the inodes, and the data?

   Answer: The image has 2000 blocks total: 31 log blocks, 13 inode blocks, and 1953 data blocks. The other three are the boot block, superblock, and bitmap block, giving 47 metadata blocks.

7 Conclusion

The experiments showed that directory names map to inodes, hard links share an inode and its data, and removal frees file data only after no links and no open references remain. The image layout and file-size limit follow directly from xv6's constants. Both new programs built and ran in QEMU; `find` found nested matches and `mystat` reflected the link-count changes.